

02_cross_cutting_foundations_ADR

ADR-0001 — Cross-Cutting Foundations (Blackwell toolchain · GPU residency · rootless CDI · unified / v1 gateway)

Type	Architecture Decision Record (foundational)
Status	ACCEPTED (cross-validated by ≥4 independent research streams)
Date	2026-07-06 · Revision 1
Track/branch	(T1/main)
Evidence basis	Streams 01 (serving), 02 (vision), 03 (translation), 04 (embeddings) independently converged; host baseline 01_host_baseline_evidence.md

These are the load-bearing decisions every capability container depends on. They are cross-validated FACT (the same constraint named by ≥3 independent streams), not a single agent’s claim (§11.4.6). Each still gets a runtime proof at build/deploy time (§11.4.108) — “the docs agree” is not “it ran on the card.”

Context

One host, one **RTX 5090 (32 GB, Blackwell sm_120)**, CUDA-12.8 driver 570.169, podman 5.7.1 rootless, 64 cores / 251 GiB RAM. The programme must run *many* local AI services (coding LLM, VLM, image-gen, video-gen, translation, embeddings, reranker, vector DB, memory, STT, OCR) and serve 6–12 concurrent agents — all in rootless-podman containers via the containers submodule.

Four independent streams surfaced the **same two hard constraints** and the **same mitigations**.

Decision 1 — Pinned Blackwell (sm_120) toolchain baseline

Every from-source GPU engine (vLLM, CTranslate2 GPU, llama.cpp CUDA, ComfyUI/torch, TEI) **MUST** be built/run against a **single pinned CUDA-12.8 toolchain**, because Blackwell sm_120 is new and prebuilt wheels frequently fail:

- **CUDA 12.8** (host driver already 12.8 ✓); base image = a pinned **CUDA 12.8 devel/runtime** image.
- **PyTorch 2.9.x + cu128** (nightly at time of research) for torch-based engines (vLLM, ComfyUI, TEI, TOWER+).
- **TORCH_CUDA_ARCH_LIST="12.0" / -DCMAKE_CUDA_ARCHITECTURES=120** for source builds.
- **FlashAttention 2**, NOT FA3 (VLLM_FLASH_ATTN_VERSION=2) — FA3 unsupported on sm_120 (streams 01, 03).
- Driver ≥ 575 recommended by stream 01 (host is 570.169 — **verify/upgrade decision needed**; flag).
- **NOT viable on consumer Blackwell (do not plan around)**: SGLang (FP8-blockwise/kernel-image errors), TensorRT-LLM (NVFP4/SM120 unwired) — stream 01. nvcc absent on host → build inside the CUDA devel container, never pollute host (G-HOST-2).

Consequence: a single tracked Dockerfile.cuda-base (or containers-submodule base spec) pins this once; every capability image FROM it. Each image ships a **real inference/embed/transcribe proof** (tok/s + nvidia-smi VRAM delta) at build — build-success alone is not acceptance (§11.4.108).

Decision 2 — Rootless GPU passthrough via NVIDIA Container Toolkit + CDI

All four streams specified the identical mechanism; host lacks it (G-HOST-1):

```
# one-time host setup (Phase-1 prerequisite; captured-proof
verified)
# install nvidia-container-toolkit (distro pkg)
nvidia-ctk cdi generate --output=$HOME/.config/cdi/nvidia.yaml
# USER-space CDI (rootless)
# per container:
podman run --device nvidia.com/gpu=all --security-
opt=label=disable <cuda-12.8 image> nvidia-smi
```

Acceptance proof (§11.4.69): podman run ... nvidia-smi inside a container prints the RTX 5090 — a real passthrough, not a config-only claim. **Known risk:**

rootless CDI can fail where privileged works (podman #17539) — fallback path documented, but rootless is the §11.4.161 target.

Decision 3 — Single-GPU VRAM residency orchestration (no naive co-hosting)

32 GB **cannot** co-host a 30B VLM + FLUX + WAN 2.2 + a 30B coder simultaneously (streams 01, 02, 04). Adopt **tiered residency**, not all-resident:

- **Always-resident:** the primary coding/agent model (Qwen3-Coder-30B-A3B, MoE, ~18 GB Q4 leaving ~12–13 GB for KV) + the small embedding model (Qwen3-Embedding-4B / can run CPU/off-peak) — the hot path for the agent fleet.
- **On-demand load/unload:** VLM, image-gen (FLUX), video-gen (LTX/WAN), TOWER+ quality-translation — loaded when a request of that class arrives, evicted under pressure. NMT (NLLB via CTranslate2 int8, ~3.5 GB) is light enough to keep warm.
- **Single-resource-owner discipline (§11.4.119):** heavy request classes serialize on the GPU; concurrent drivers of the same VRAM would cross-contaminate/ OOM. A VRAM budget-broker gates admission.
- **Per-engine memory caps** (--gpu-memory-utilization, llama.cpp --parallel/ctx caps) enforce the budget; §11.4.133 target-safety — never exceed the card's safe envelope, capture thermal evidence under sustained load.

Consequence: HelixLLM/HelixAgent need a **model-residency scheduler / VRAM broker** (a new component) that admits/evicts model containers by request class and a VRAM budget. This is a first-class Phase-1/3 design item.

Decision 4 — One unified OpenAI/Anthropic-compatible /v1 gateway surface

Streams 02 + 03 (and the inventory) independently arrived at a **LiteLLM-style single /v1 gateway**: every capability (chat, vision chat, /v1/images/generations, async video jobs, /v1/embeddings, rerank, /translate, /v1/audio/transcriptions, OCR) is presented behind ONE OpenAI/Anthropic-compatible surface with a single /v1/models list — so **Claude Code / HelixCode auto-recognize everything**. HelixAgent already ships an OpenAI/Anthropic/Google-compatible server (inventory) → **extend it as this gateway** rather than add a new one (§11.4.74 reuse-don't-reimplement). Non-standard surfaces (video gen) are exposed as async job endpoints + advertised via capability flags (CONST-040 / stream 05).

Converged risk register (feeds stream 99)

ID	Risk (streams that named it)	Mitigation
CX-01	Blackwell sm_120 build friction — prebuilt wheels fail (01,02,03,04)	pinned CUDA-12.8 devel container + source builds + per-image runtime proof
CX-02	32 GB VRAM contention — can't co-host all services (01,02,04)	tiered residency + VRAM broker + §11.4.119 single-owner serialization
CX-03	Rootless CDI GPU passthrough may fail where privileged works (02,03,04)	verified nvidia-smi-in-container proof; documented fallback
CX-04	Benchmark/name drift — many 2026 tok/s figures + model names are aggregator-only (01,02,03,04)	every headline number re-measured on the actual card before commit (§11.4.6); UNCONFIRMED names flagged
CX-05	Metric-gaming (COMET contamination in translation; MTEB in embeddings) (03,04)	QE + human/MQM spot-checks; verify against model cards/primary papers
CX-06	License traps (Jina reranker CC-BY-NC weights) (04)	ship only Apache/MIT weights self-hosted
CX-07	Large weights unversioned → fresh clone can't run (all)	§11.4.77 fetch_weights.sh + .gitignore-meta re-obtain, wired into setup.sh

Model selections locked by evidence (subject to on-card re-measure)

Capability	Primary	Serving	Approx VRAM	Notes
Coding+agent (fleet)	Qwen3-Coder-30B-A3B-Instruct (MoE)	vLLM (sm_120 build) / llama-server fallback	~18 GB Q4 + KV	12+ agents @16k; ~1/3 KV of dense 32B
Heavy-coding lane	Qwen2.5-Coder-32B	llama-server	~22 GB	low-parallel; HumanEval 88.4
Agent tool-use lane	Devstral-Small-2507 (24B)	vLLM	~	SWE-bench Verified 53.6%
Vision (VLM)	Qwen3-VL-8B (30B high-q)	vLLM / llama.cpp+mmpromj	~12–16 GB	on-demand
Image gen	FLUX.1-dev fp8 / schnell	ComfyUI	on-demand	Blackwell FP8
Video gen	LTX-Video/ LTX-2 + WAN 2.2 14B fp8	ComfyUI	on-demand	both fit 32 GB ✓
Vectorize	vtracer (+ StarVector-8B)	FastAPI / VLM	CPU / on-demand	raster → SVG
Translation NMT	NLLB-200-3.3B	CTranslate2 int8	~3.5 GB	warm; 200 langs
Translation quality	TOWER+ 9B	vLLM	~18 GB	on-demand; XCOMET 84.38
Embedding	Qwen3-Embedding-4B (+BGE-M3 sparse)	TEI / Infinity	~8 GB	general+code
Reranker	bge-reranker-v2-m3	TEI	small	Apache-2.0
Vector DB	Qdrant	podman	CPU	hybrid dense+sparse

Capability	Primary	Serving	Approx VRAM	Notes
Memory	Zep/Graphiti + mem0	podman	CPU	HelixMemory spine, MCP-exposed

Every row above is a research recommendation with a cited source in the stream reports; each becomes a runtime-proven FACT only after an on-card measurement at build/deploy (§11.4.108 / §11.4.123).